

MODULE 4 — OBJECT SERIALIZATION AND DESERIALIZATION IN JAVA

Learning Objectives

By the end of this module, students should be able to:

1. Understand the concept of **serialization** and **deserialization** in Java.
2. Know why and when to use object serialization.
3. Learn how to serialize and deserialize objects using `ObjectOutputStream` and `ObjectInputStream`.
4. Handle exceptions during object file operations.
5. Implement `transient` and `serialVersionUID` in serializable classes.
6. Combine object streams with the `java.nio.file` package.

4.1 Introduction to Serialization

What is Serialization?

Serialization is the process of **converting an object's state into a byte stream** so it can be:

- Stored in a file or database,
- Sent over a network,
- Or saved for later use.

The reverse process, converting the byte stream back into an object, is called **Deserialization**.

◆ Why Serialize Objects?

Purpose	Example
Saving program state	Save game progress or application settings
Network transfer	Send Java objects between clients and servers
Caching	Store preprocessed data for quick access later
Persistence	Store objects permanently in a file



4.2 The Serializable Interface

In Java, any class that wants its objects to be serializable must **implement the Serializable interface**.



Key Points

- Serializable is a **marker interface** — it has **no methods**.
- It tells the JVM that the object's data can be written to a stream.



Example 1: Simple Serializable Class

```
import java.io.Serializable;

public class Student implements Serializable {
    private String name;
    private int age;

    public Student(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public void display() {
        System.out.println("Name: " + name + ", Age: " + age);
    }
}
```

✅ This class is now serializable because it implements `Serializable` .

4.3 Writing Objects to a File (Serialization)

We use:

- `FileOutputStream` → writes raw bytes to a file
- `ObjectOutputStream` → converts Java objects to bytes



Example 2: Serialize an Object

```
import java.io.*;

public class SerializeExample {
    public static void main(String[] args) {
        Student student = new Student("Alice", 21);

        try {
            FileOutputStream fileOut = new FileOutputStream("student.ser");
            ObjectOutputStream out = new ObjectOutputStream(fileOut);

            out.writeObject(student);
            out.close();
            fileOut.close();

            System.out.println("Object has been serialized and saved to student.ser");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Explanation:

- `writeObject()` converts the `Student` object into bytes.
- `.ser` is a common file extension for serialized objects.
- Always close the streams to release resources.

4.4 Reading Objects from a File (Deserialization)

We use:

- `FileInputStream` → reads raw bytes from the file
- `ObjectInputStream` → converts bytes back to a Java object



Example 3: Deserialize an Object

```
import java.io.*;

public class DeserializeExample {
    public static void main(String[] args) {
        try {
            FileInputStream fileIn = new FileInputStream("student.ser");
            ObjectInputStream in = new ObjectInputStream(fileIn);

            Student student = (Student) in.readObject();
            in.close();
            fileIn.close();

            System.out.println("Object has been deserialized:");
            student.display();
        } catch (IOException | ClassNotFoundException e) {
            e.printStackTrace();
        }
    }
}
```

Explanation:

- `readObject()` returns an `Object`, so you must cast it to `Student`.
- Both the writing and reading classes must have compatible class definitions.

4.5 Using `serialVersionUID`

`serialVersionUID` is a **unique identifier** for each `Serializable` class.

It ensures that a serialized object matches the version of the class used to deserialize it.



Example 4: Using serialVersionUID

```
import java.io.Serializable;

public class Employee implements Serializable {
    private static final long serialVersionUID = 1L;

    private String name;
    private double salary;

    public Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }

    public void show() {
        System.out.println(name + " earns $" + salary);
    }
}
```

Why Important:

If the class definition changes (e.g., you add/remove fields) but don't change `serialVersionUID`, Java can throw an `InvalidClassException` during deserialization.



4.6 The transient Keyword

The `transient` keyword tells Java **not to serialize** a specific field. It's used for sensitive or temporary data that shouldn't be saved.



Example 5: Using transient

```
import java.io.Serializable;

public class User implements Serializable {
    private String username;
    private transient String password; // Will not be serialized

    public User(String username, String password) {
        this.username = username;
        this.password = password;
    }

    public void display() {
        System.out.println("Username: " + username + ", Password: " + password);
    }
}
```

When deserialized, password will be null (default value) because it wasn't serialized.



4.7 Serializing Multiple Objects

You can serialize multiple objects by writing them one after another.



Example 6: Writing Multiple Objects

```
import java.io.*;

public class MultiSerialize {
    public static void main(String[] args) {
        try (ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream("students.ser"))) {
            out.writeObject(new Student("Alice", 21));
            out.writeObject(new Student("Bob", 23));
            out.writeObject(new Student("Charlie", 19));
            System.out.println("Multiple objects serialized!");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```



Example 7: Reading Multiple Objects

```
import java.io.*;

public class MultiDeserialize {
    public static void main(String[] args) {
        try (ObjectInputStream in = new ObjectInputStream(new FileInputStream("students.ser"))) {
            for (int i = 0; i < 3; i++) {
                Student s = (Student) in.readObject();
                s.display();
            }
        } catch (IOException | ClassNotFoundException e) {
            e.printStackTrace();
        }
    }
}
```



4.8 Serializing Collections

You can serialize whole **lists or maps** if their elements are also serializable.



Example 8: Serializing an ArrayList

```
import java.io.*;
import java.util.*;

public class ListSerialize {
    public static void main(String[] args) {
        List<Student> list = new ArrayList<>();
        list.add(new Student("Alice", 21));
        list.add(new Student("Bob", 22));
        list.add(new Student("Eve", 20));

        try (ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream("studentList
            out.writeObject(list);
            System.out.println("ArrayList serialized!");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```



Example 9: Deserializing an ArrayList

```
import java.io.*;
import java.util.*;

public class ListDeserialize {
    public static void main(String[] args) {
        try (ObjectInputStream in = new ObjectInputStream(new FileInputStream("studentList.ser'
            List<Student> list = (List<Student>) in.readObject();
            for (Student s : list) {
                s.display();
            }
        } catch (IOException | ClassNotFoundException e) {
            e.printStackTrace();
        }
    }
}
```


4.9 Best Practices

Practice	Description
Use <code>serialVersionUID</code>	Ensures version compatibility
Mark sensitive fields <code>transient</code>	Protects data like passwords
Always close streams	Prevents resource leaks
Use <code>try-with-resources</code>	Simplifies file handling
Handle exceptions	Catch <code>IOException</code> and <code>ClassNotFoundException</code>

4.10 Classwork / Practical Exercises

1. Create a class `Person` with name, email, and age. Serialize and deserialize the object.
2. Modify the class by adding a new field (e.g., phone number). Re-run deserialization and observe the effect.
3. Create a class `Account` with username and password (password should be transient). Test serialization and check results.
4. Create and serialize a `List` of `Person` objects. Deserialize and print all.
5. Save multiple `Employee` objects in a file and retrieve them.

4.11 Challenge Project

Create a mini **Contact Management System** that:

- Allows users to create new contacts (name, phone, email).
- Saves all contacts to a serialized file (`contacts.ser`).
- Loads and displays all contacts when reopened.
- Automatically updates the list when a new contact is added.

Hint: Use `ArrayList<Contact>` and serialize the entire list after every update.



4.12 Summary of Module 4

Concept	Class/Keyword	Description
Serialization	<code>ObjectOutputStream</code>	Converts objects to byte stream
Deserialization	<code>ObjectInputStream</code>	Converts byte stream back to object
Serializable	<code>implements Serializable</code>	Marks class as serializable
<code>transient</code>	<code>transient</code> keyword	Excludes a field from serialization
<code>serialVersionUID</code>	<code>private static final long</code>	Ensures version compatibility
<code>try-with-resources</code>	<code>try (...)</code>	Auto-closes streams

🏁 End of Module 4 Summary

In this final module, you learned:

- How to serialize and deserialize Java objects.
- The use of `transient` and `serialVersionUID`.
- How to persist multiple and collection-based objects efficiently.

With this, you now have a **complete mastery of File Handling in Java** — from simple text files to advanced object persistence.